



۸۱۰۱۰۱۴۰۸

۸۱۰۱۰۱۳۹۳

شبکه‌های کامپیوتری

نام و نام خانوادگی: بابک حسینی محتشم - پریا پاسه‌ورز



فاز ۲ پروژه ۱

## گزارش فاز دوم پروژه اول

### کدها

ابتدا برنامه‌های نوشته شده برای این پروژه را بررسی می‌کنیم، سپس نتایج آنها را تحلیل می‌کنیم:

`phase2.tcl`

این اسکریپت برای مقایسه عملکرد پروتکل‌های TCP Reno و TCP Tahoe و TCP Vegas در یک شبکه دارای bottleneck و تأخیر متغیر طراحی شده است. دو جریان TCP هم‌زمان از یک گلوگاه مشترک عبور می‌کنند و معیارهایی مانند RTT (Round Trip Time) و میزان داده دریافتی Throughput و رفتار در مواجهه با ازدحام در طول ۱۰۰۰ ثانیه شبیه‌سازی ثبت می‌شود.

آرگومان‌های ورودی این برنامه عبارتند از نوع TCP مورد آزمایش (Tahoe, Reno, Vegas)، seed (بذر تصادفی برای تکرارپذیری نتایج) و پیشوند فایل‌های خروجی است. با پیشوند فایل‌های خروجی، تأخیرهای تصادفی لینک‌ها را خواهیم داشت.

برای رسم نمودارهای throughput و RTT در آینده، تعدادی فایل باید تولید کنیم:

- `tr`: این فایل تریس کامل شبیه‌سازی را در خود ذخیره می‌کند.
- `q.tr`: این فایل تریس صف گلوگاه را خواهد داشت.
- `rtt-f1.dat`: این فایل RTT را برای جریان اول در خود دارد.
- `rtt-f2.dat`: این فایل RTT را برای جریان دوم در خود دارد.
- `bytes-f1.dat`: این فایل بایت دریافتی را برای جریان اول در خود دارد.
- `bytes-f2.dat`: این فایل بایت دریافتی را برای جریان دوم در خود دارد.

اندازه صف گلوگاه در این شبیه‌سازی ۱۰ بسته و اندازه TCP payload ۱۰۰۰ بایت است. لینک‌ها و گلوگاه‌ها مشابه شکل در صورت پروژه تعریف شده‌اند و هر 0.5 ثانیه، تأخیر لینک‌های  $n2 - rL$  و  $rR - n6$  به صورت تصادفی تغییر می‌کند.

برای لاگ گرفتن از RTT و بایت دریافتی، هر ۱ ثانیه مقدار srtt از TCP و مقدار bytes از TCPSink خوانده می‌شود.

**run all.sh**

این اسکریپت وظیفه اجرای خودکار و تکرارشونده شبیه‌سازی NS2 برای چند نوع پروتکل TCP و چند مقدار بذر تصادفی بر عهده دارد. هدف آن جمع‌آوری نتایج آماری قابل اعتماد از طریق اجرای چندباره هر سناریو است.

**make dat.py**

این اسکریپت برای پردازش خروجی شبیه‌سازی‌ها است. برای این کار در حلقه اصلی sum tp, sum rtt, sum loss مقادیرشان در هر ثانیه جمع می‌شود و در نهایت بر اساس تعداد اجراها میانگین‌گیری می‌شود.

**detect payload rule from trace**

برای تشخیص خودکار اندازه payload از روی trace، باید در نظر گرفت ستون size گاهی شامل header payload و گاهی شامل فقط payload است. به همین دلیل اندازه‌های کوچک را TCP header و اندازه‌های بزرگ را TCP data در نظر می‌گیریم. سپس به صورت آماری تشخیص می‌دهیم که آیا  $payload = sizeheader$  یا  $payload = size$  صحیح است.

**throughput from trace payload mbps**

برای محاسبه Throughput فقط بسته‌های از نوع TCP که دریافت شده‌اند و به گره مقصد نهایی نیز رسیده‌اند در نظر گرفته‌شوند. سپس:

$$Throughput(t) = \frac{PayloadBytes(t) \times 8}{10^6} Mbps$$

**parse queue trace loss percent**

در محاسبه نرخ packet loss از فایل q.tr از علامت + ورود بسته‌ها به صف را متوجه می‌شویم و از d بسته‌هایی که drop شده‌اند. سپس loss با توجه به رابطه زیر محاسبه می‌شود:

$$Loss = 100 \times \frac{Drops}{Enqueues}$$

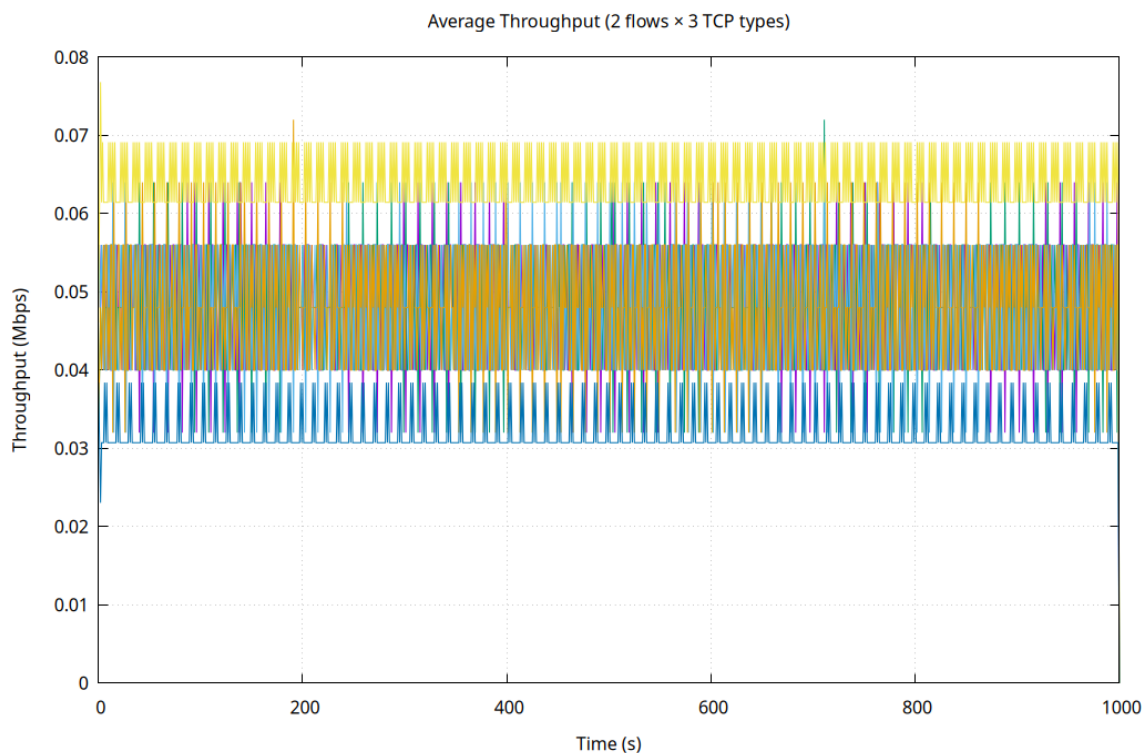
و در نهایت نتایج روی ۱۰ اجرا تجمیع می‌شوند فایلهای throughput avg.dat, rtt avg.dat, loss avg.dat تولید می‌شوند.

## نتایج

در این قسمت نمودارها برای هر پروتکل و هر Flow رسم می‌شوند:

### Throughput

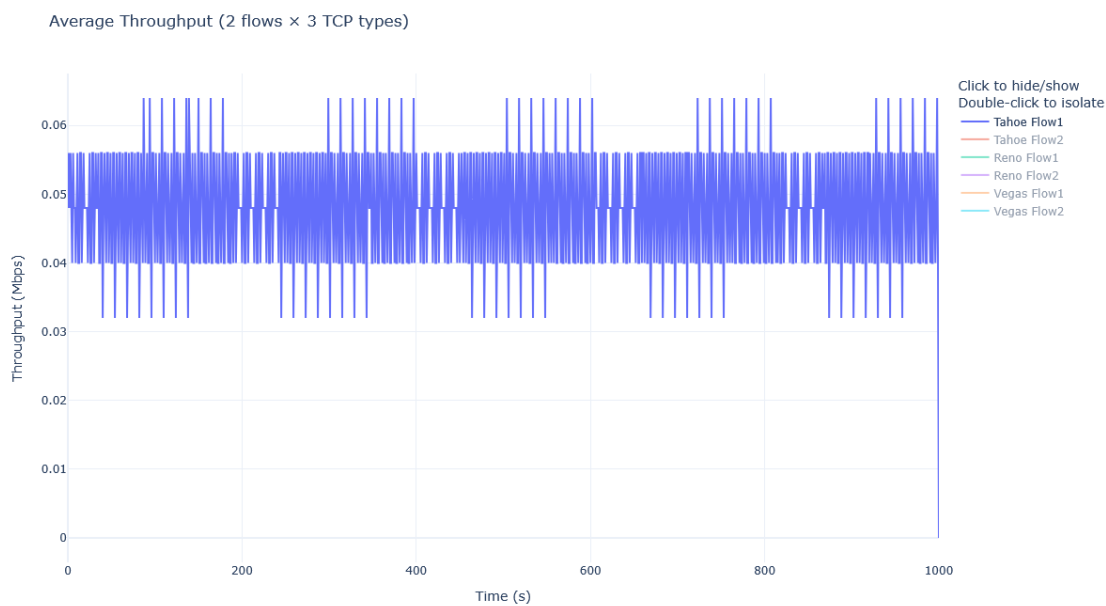
شکل ۱: نمودار throughput برای حالات مختلف



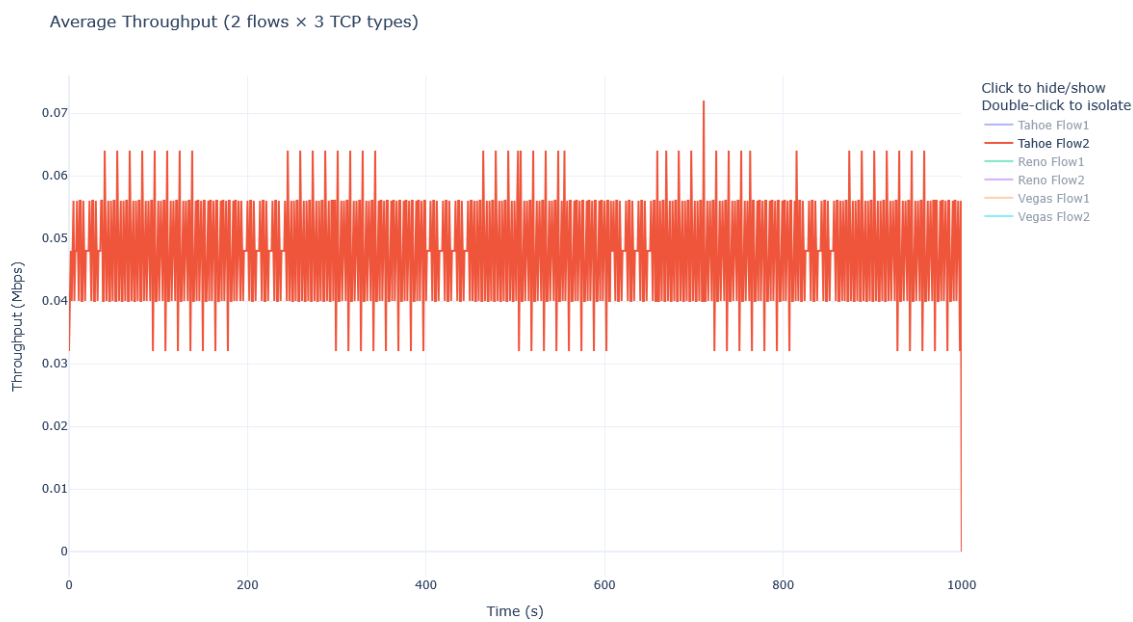
برای بررسی بهتر، آن را برای هر Flow و هر پروتکل جداگانه رسم می‌کنیم:

### Throughput, Tahoe

سطح کلی نمودار برای هر دو Flow تقریباً یکسان است و این یعنی هر دو تقریباً عادلانه تقسیم پهنای باند کرده‌اند، تقریباً به اندازه نصف bottleneck. در هر دو نوسان‌ها به شکل دندان‌های/پالسی دیده می‌شوند، ولی در Flow2 کمی پیک‌های بزرگ‌تر مشاهده می‌کنیم که می‌تواند ناشی از delay تصادفی در مسیر Flow2 باشد. هر دو نمودار نشان می‌دهند Tahoe بعد از پر شدن صف/ازدحام، بالا و پایین می‌شود که رفتاری طبیعی است.



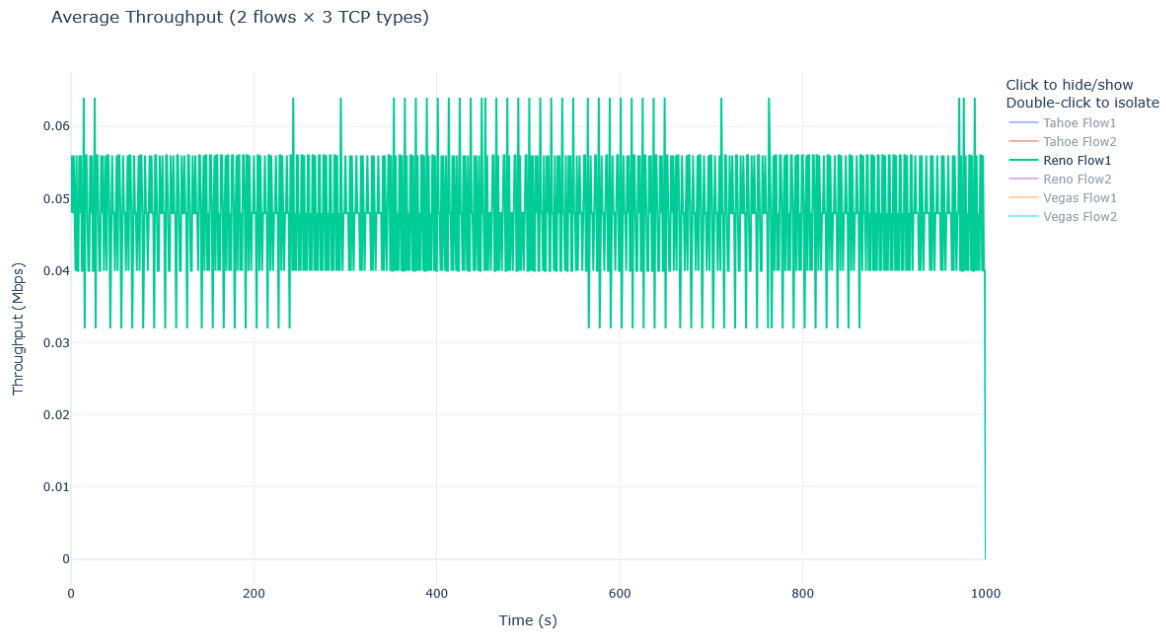
شکل ۲: نمودار throughput برای Flow اول از روش Tahoe



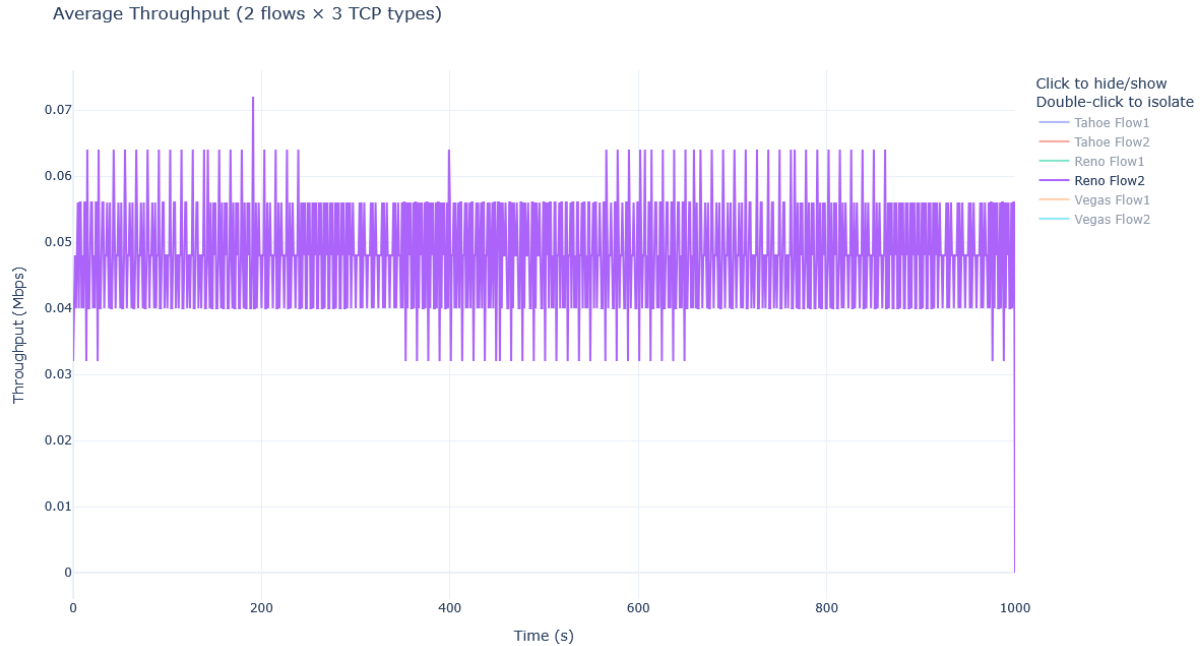
شکل ۳: نمودار throughput برای Flow دوم از روش Tahoe

## Throughput, Reno

در این روش، بازه اصلی دو نمودار تقریباً یکسان است یعنی از نظر میانگین کلی خیلی تفاوت بزرگی ندارد. در Flow اول کلیت نمودار نسبتاً پایدارتر به نظر می‌رسد (کمتر وارد افت‌های عمیق و طولانی می‌شود) و چند قله‌ی کوتاه نزدیک هم دیده می‌شود در حالی که در Flow دوم نوسان‌های شدیدتر داریم یک قله‌ی بزرگ دیده می‌شود. مشابه قبل، دلیل است که در Flow دوم delay به صورت تصادفی تغییر می‌کند.



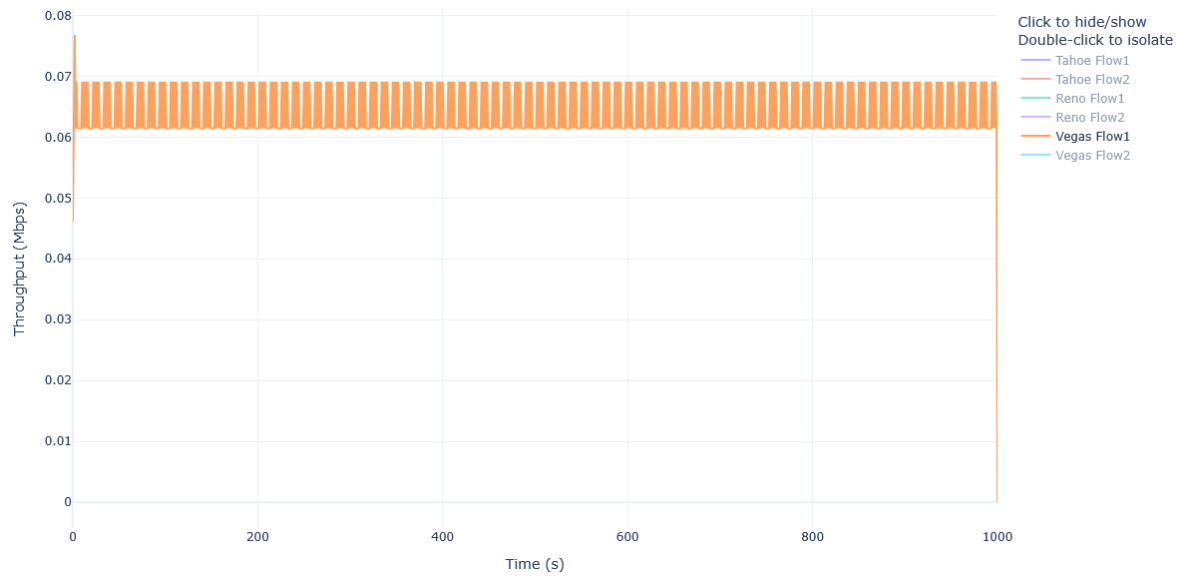
شکل ۴: نمودار throughput برای Flow اول از روش Reno



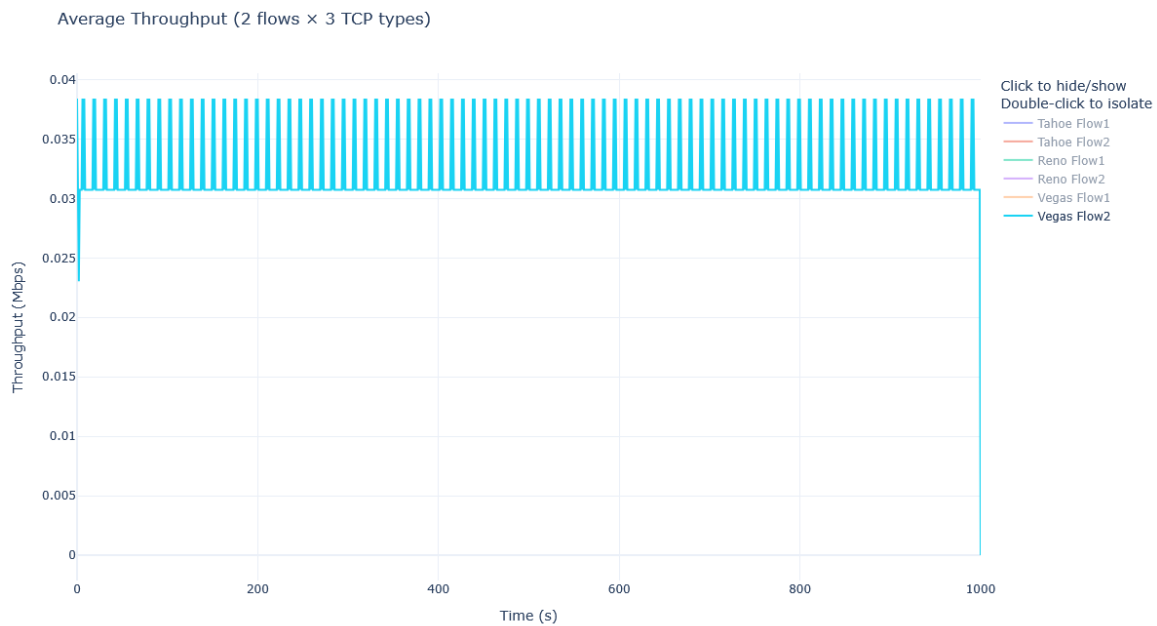
شکل ۵: نمودار throughput برای Flow دوم از روش Reno

## Throughput, Vegas

در این روش در نمودار اول throughput تقریباً ثابت و پایدار است و شکل نوسان‌ها خیلی منظم و کوچک است، یعنی Flow اول سهم بیشتری از bottleneck را گرفته است. در حالی که در Flow دوم throughput پایین‌تر است و نوسان‌ها باز هم منظم و کم هستند، ولی سطح کلی تقریباً نصف Flow اول است، یعنی Flow دوم سهم کمتری از bottleneck می‌گیرد. پس نتیجه می‌گیریم Vegas بین دو جریان تقسیم کاملاً مساوی انجام نداده است. اما Vegas نسبت به Tahoe/Reno خیلی صاف‌تر و پایدارتر است که با ماهیت delay-based بودن آن همخوانی دارد.



شکل ۶: نمودار throughput برای Flow اول از روش Vegas



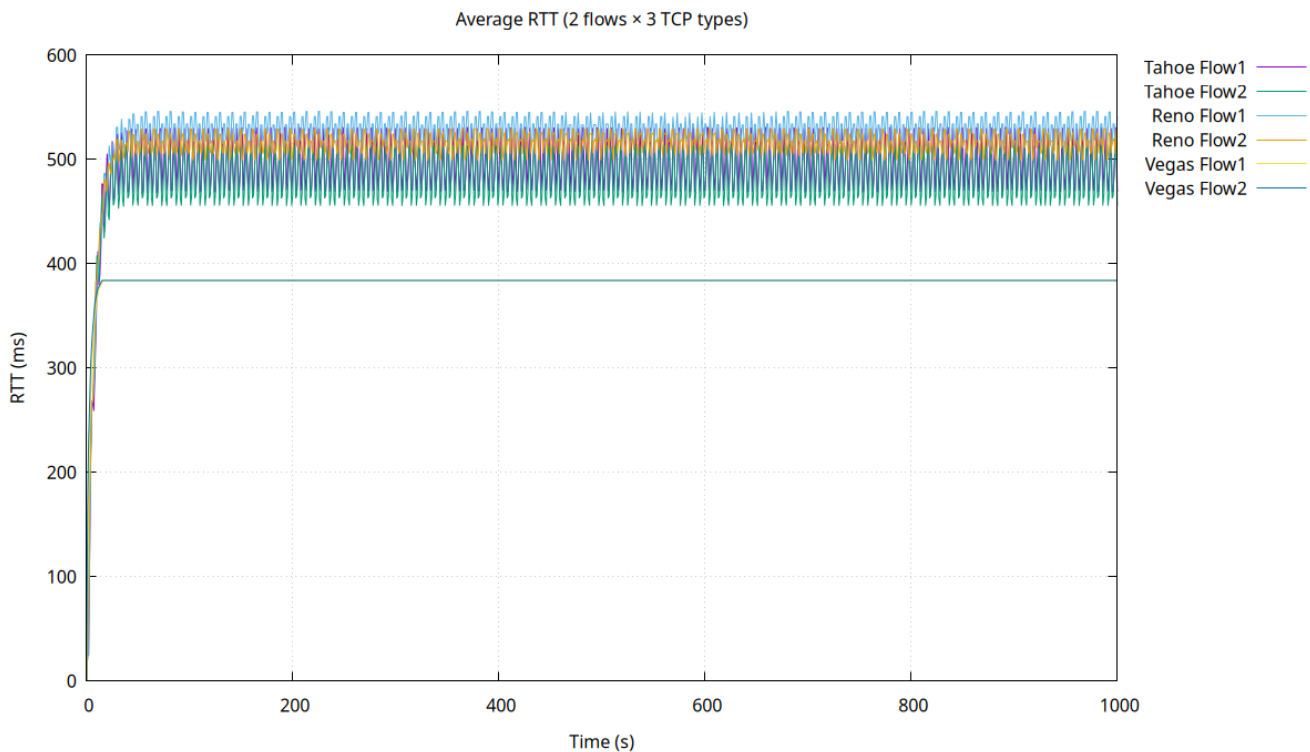
شکل ۷: نمودار throughput برای Flow دوم از روش Vegas



در TCP Vegas به دلیل حساسیت به RTT و وجود تأخیر تصادفی در مسیر Flow2، نرخ Flow2 محافظه‌کارانه‌تر شده و سهم بیشتری از ظرفیت گلوگاه را گرفته است؛ در عوض نوسانات throughput نسبت به Reno/Tahoe کمتر و رفتار پایدارتر است.

## RTT

شکل ۸: نمودار RTT برای حالات مختلف



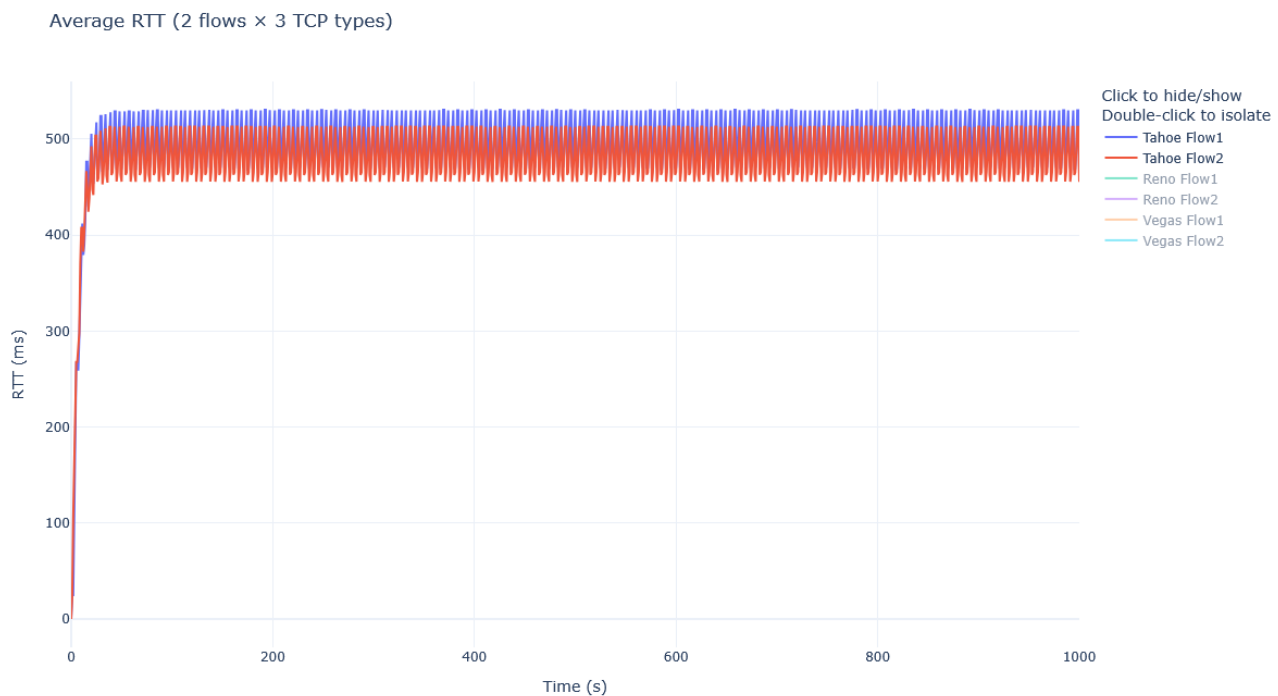
برای بررسی بهتر، آن را برای هر Flow و هر پروتکل جداگانه رسم می‌کنیم:

## RTT, Tahoe

Flow اول تقریباً یک باند بالاتر از Flow دوم قرار دارد. هر دو بعد از چند ثانیه‌ی اول، وارد حالت پایدار می‌شوند و نوسان دندانه‌اره‌ای دارند و این برای Tahoe طبیعی است چون با افزایش پنجره ازدحام و سپس افت ناگهانی بعد از ازدحام/ازدست‌دادن، صف گلوگاه پر و خالی می‌شود و RTT بالا و پایین می‌رود. نوسان‌ها در هر دو جریان مشابه‌اند، ولی چون Flow اول بالاتر نشسته، یعنی به‌طور متوسط صف بیشتری را تجربه می‌کند.

اگر Flow اول تهاجمی‌تر/سریع‌تر بسته تزریق کند سهم بیشتری از صف را اشغال می‌کند و بسته‌های خودش بیشتر پشت صف می‌مانند، به همین دلیل RTT در این Flow بیشتر دیده می‌شود. اما در Flow دوم با RTT پایه بزرگ‌تر (به خاطر دیلی رندوم) ACK ها دیرتر برمی‌گردند، ها دیرتر برمی‌گردند، کمتر صف را پر می‌کند و حتی ممکن است وقتی صف خلوت‌تر است برسد. به همین دلیل مجموع RTT می‌تواند پایین‌تر شود.

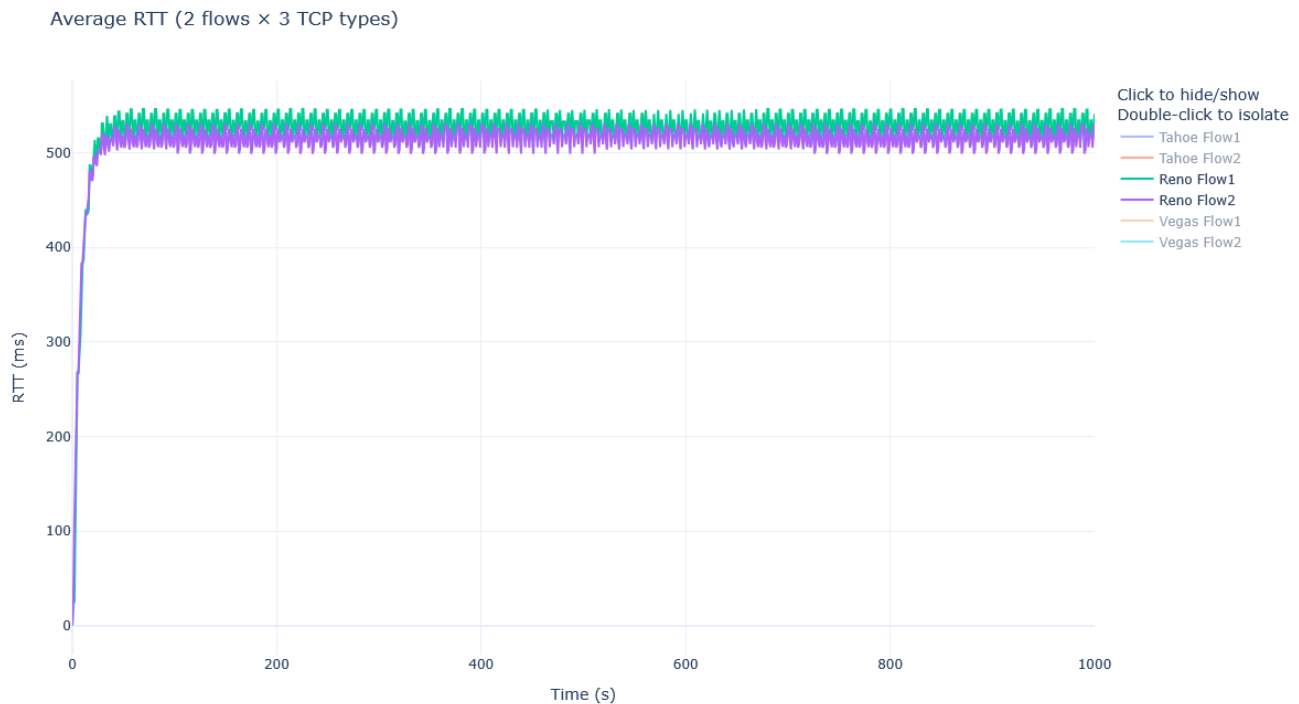
شکل ۹: نمودار RTT برای روش Tahoe



## RTT, Reno

RTT در این روش نیز تقریباً همیشه در Flow اول بالاتر از دوم قرار دارد. هر دو در چند ثانیه/ده‌ثانیه‌ی اول سریع بالا می‌روند و به حالت پایدار می‌رسند (پر شدن صف گلوگاه + شروع کنترل ازدحام).  
نوسان‌ها دندانه‌ای و منظم هستند که برای Rebo طبیعی است، ولی دامنه‌ی نوسان Flow اول کمی بیشتر به نظر می‌رسد. در این سناریو که Flow دوم که روی مسیرش دیلی رندوم دارد باعث می‌شود این مسیر کمتر تهاجمی باشد.

شکل ۱۰: نمودار RTT برای روش Reno



## RTT, Vegas

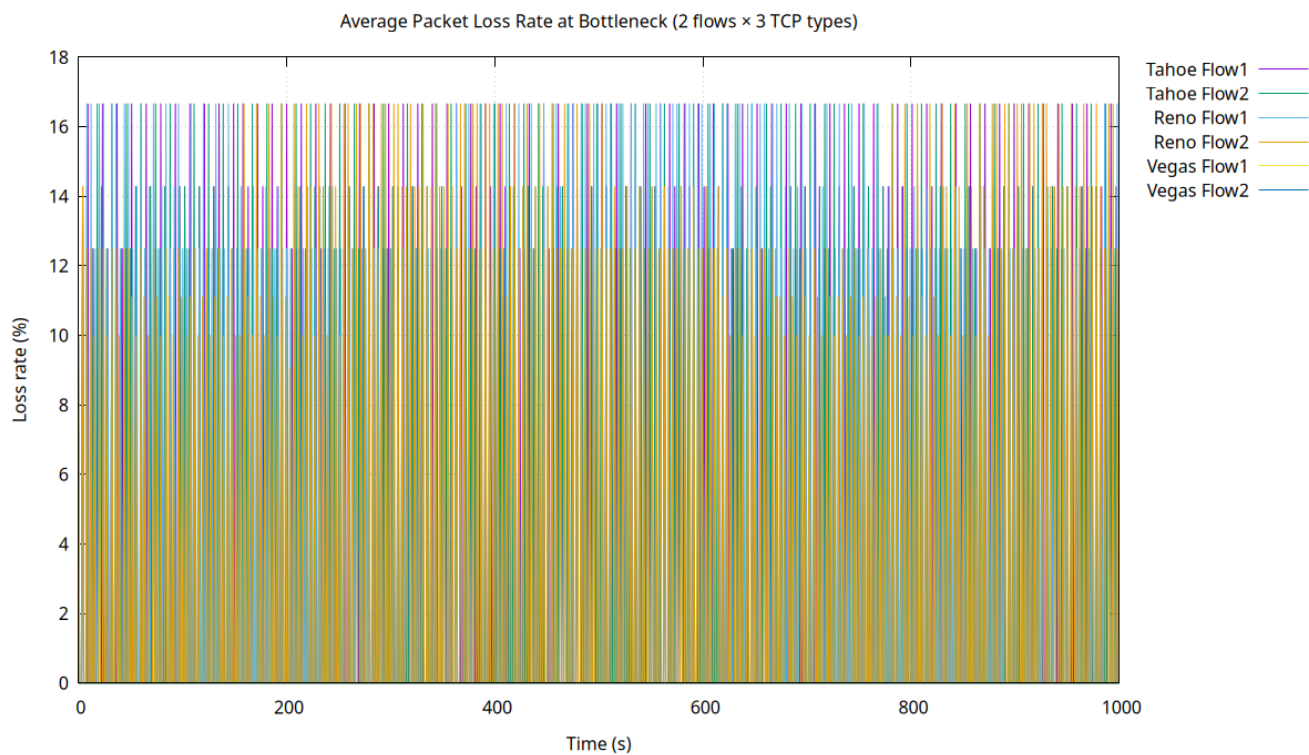
شکل ۱۱: نمودار RTT برای روش Vegas



هر دو جریان خیلی سریع (در چند ثانیه‌ی اول) به حدود  $380 \pm 385$  ms می‌رسند و بعد تقریباً کاملاً ثابت می‌مانند. چون دو منحنی روی هم هستند، اختلاف RTT بین دو فلو خیلی ناچیز است. بر خلاف Tahoe/Reno که دندانه‌اره‌ای و پرنوسان بودند، Vegas اینجا RTT را تقریباً ثابت نگه داشته است. این نکته با ماهیت Vegas تطابق دارد، چون Vegas delay based است و سعی می‌کند قبل از پر شدن صف، نرخ را تنظیم کند تا صف گلوگاه زیاد بالا و پایین نشود.

## Loss

شکل ۱۲: نمودار Loss برای حالات مختلف

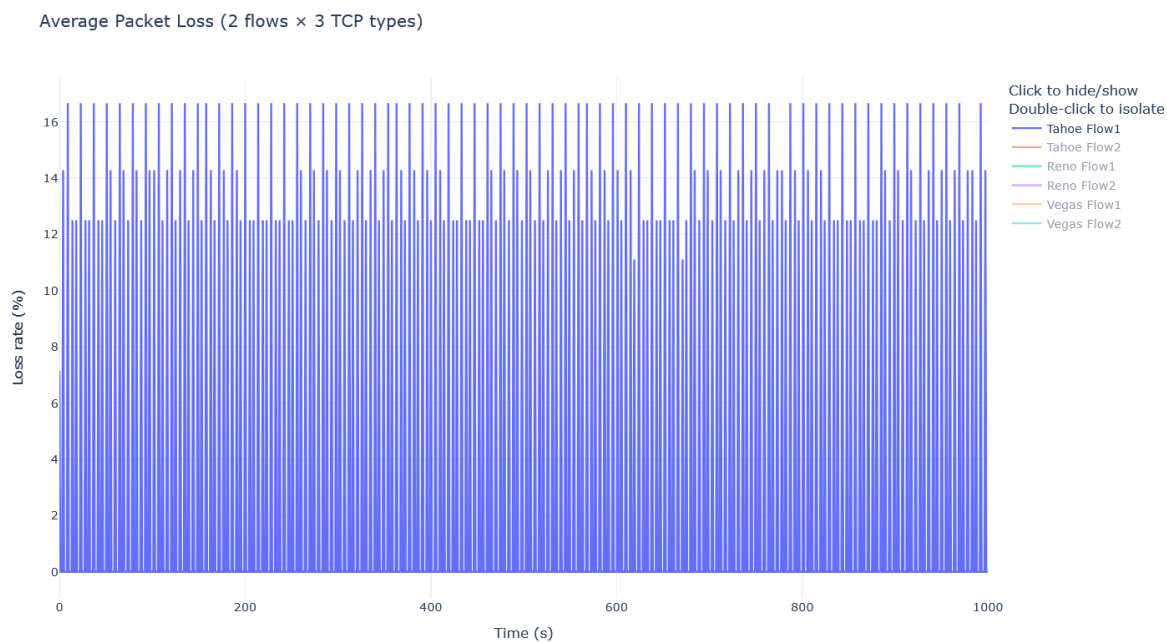


برای بررسی بهتر، آن را برای هر Flow و هر پروتکل جداگانه رسم می‌کنیم:

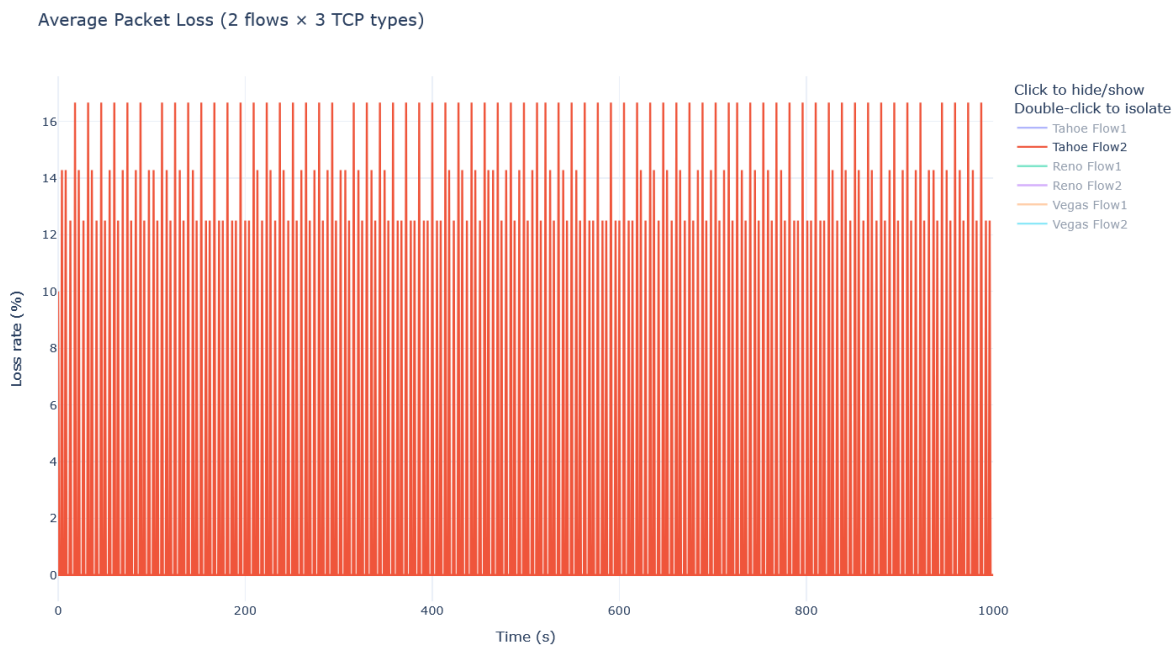
## Loss, Tahoe

هر دو نمودار یک الگوی میله‌ای/پالسی دارند و مرتباً بین و مرتباً بین ۰ درصد و یک‌سری مقدارهای ثابت بالا پایین می‌روند. این رفتار طبیعی است چون در هر ثانیه تعداد بسته‌های وارد صف محدود است در FLOW اول به طور کلی حس می‌شود کمی نرم‌تر/کم‌نوسان‌تر از FLOW دوم است، یعنی FLOW دوم کمی بیشتر دچار Drop های شدید (درصدهای بالاتر) می‌شود یا دست‌کم بیشتر وارد وضعیت‌هایی می‌شود که در یک ثانیه تعداد drop نسبت به enqueue زیاد است.

با وجود اینکه هر دو روی یک گلوگاه مشترک‌اند، FLOW دوم چون مسیرش delay رندوم دارد، ممکن است burst های زمانی متفاوت‌تری تولید کند یا در بعضی ثانیه‌ها دقیقاً وقتی صف پرتر است به گلوگاه برسد.



شکل ۱۳: نمودار Loss برای Flow اول از روش Tahoe

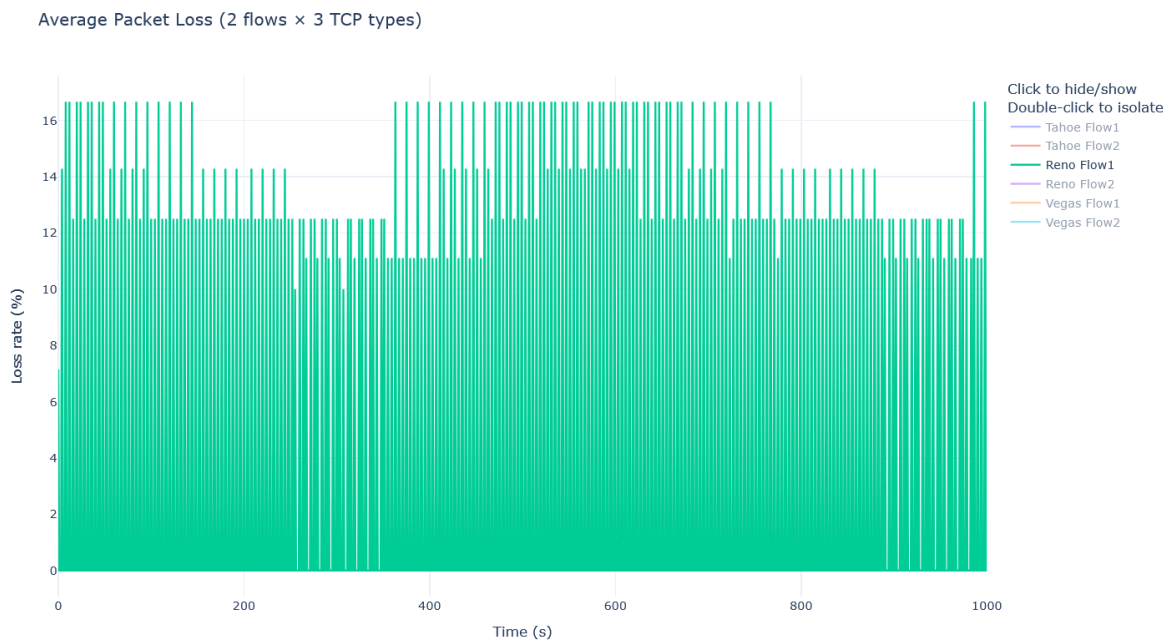


شکل ۱۴: نمودار Loss برای Flow دوم از روش Tahoe

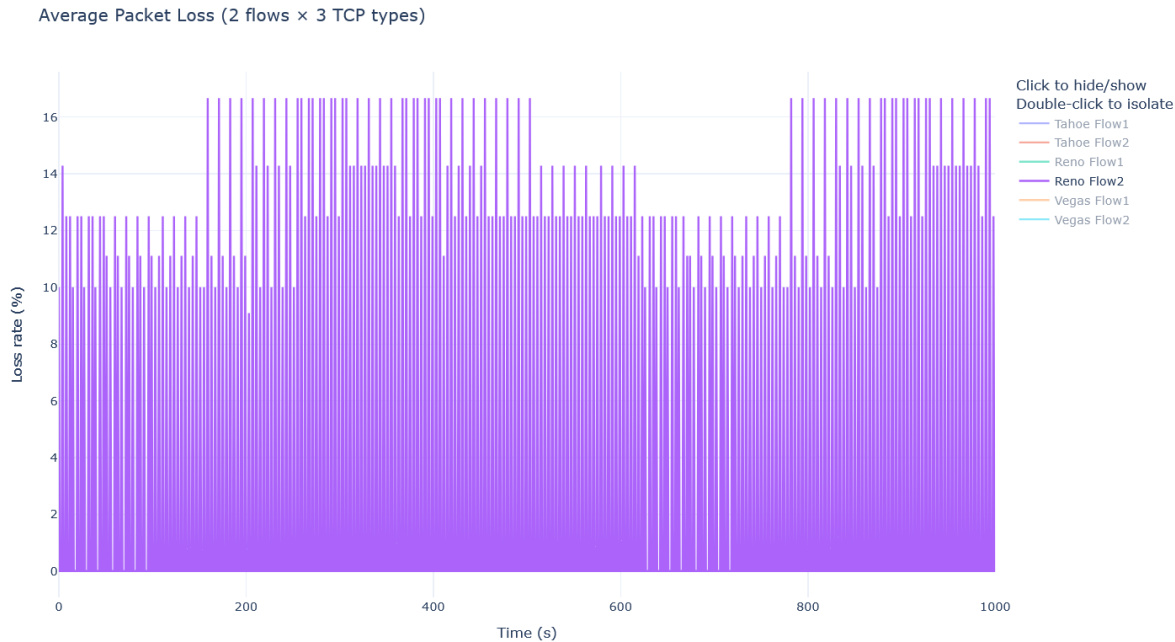
## Loss, Reno

در TCP Reno، هر دو جریان به دلیل DropTail و صف کوچک گلوگاه، loss نسبتاً بالا و پله‌ای دارند. با این حال Flow1 معمولاً در یک بازه‌ی نسبتاً پایدارتر (حدود  $10^4$  تا  $10^5$ ) قرار دارد، در حالی که Flow2 به دلیل تغییرپذیری مسیر (random delay) نوسانات زمانی بیشتری نشان می‌دهد؛ به‌طوری که در برخی بازه‌ها loss کمتر و در برخی بازه‌ها پیک‌های شدیدتر و پرتکرارتر دارد.





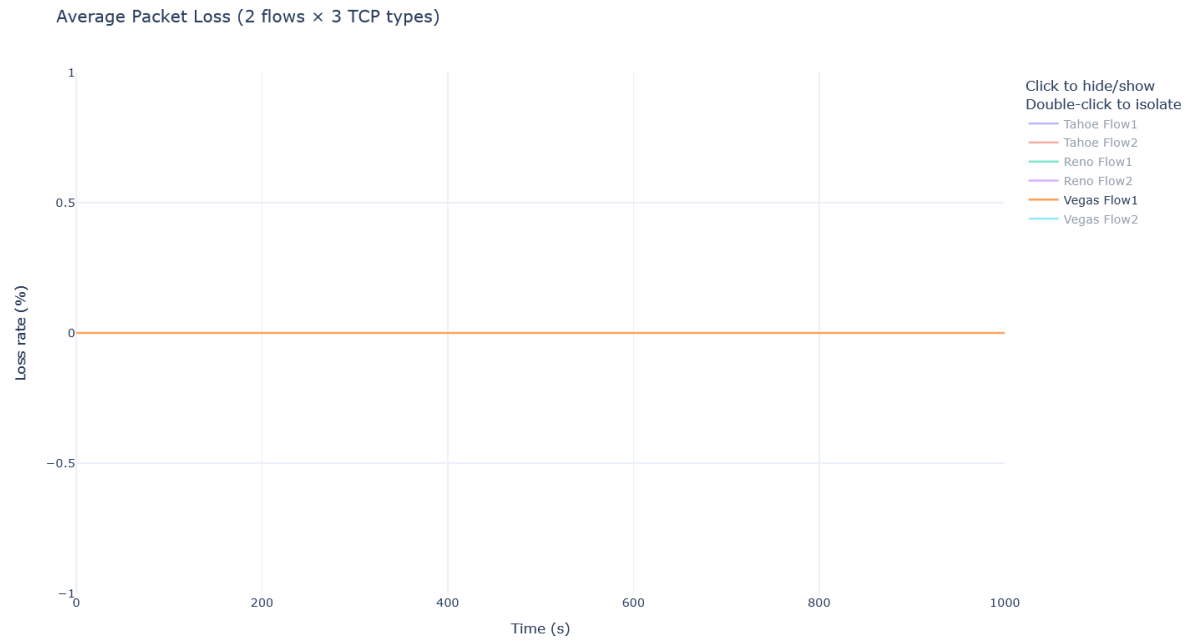
شکل ۱۵: نمودار Loss برای Flow اول از روش Reno



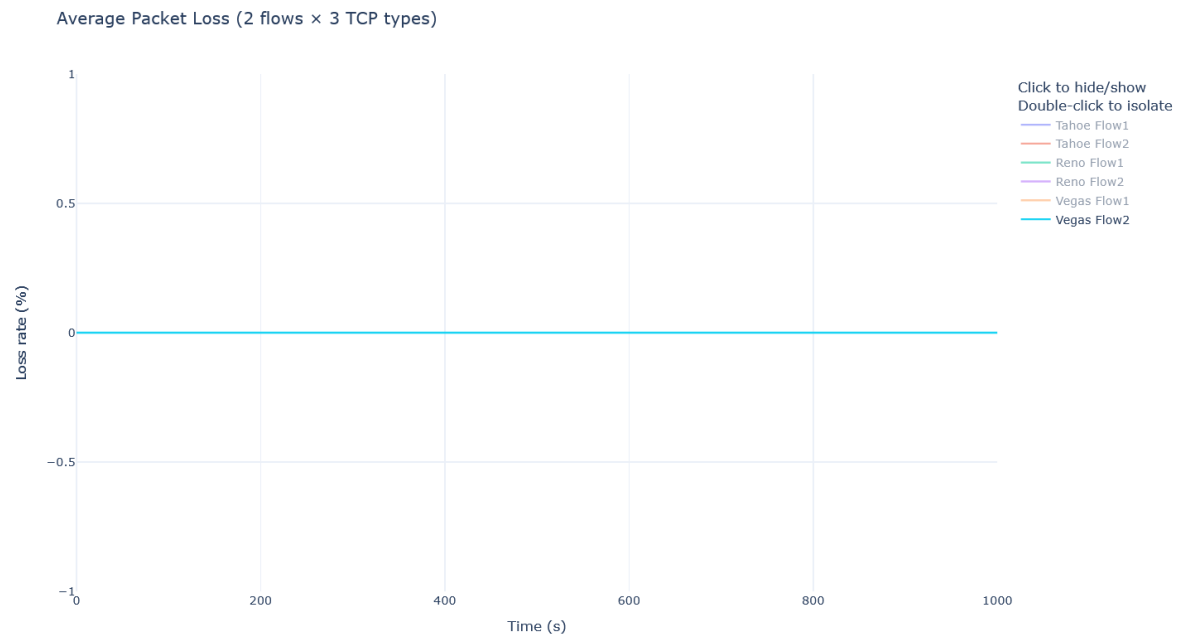
شکل ۱۶: نمودار Loss برای Flow دوم از روش Reno

## Loss, Vegas

در TCP Vegas برای هر دو جریان نرخ از دست رفتن بسته تقریباً صفر است، زیرا Vegas با کنترل ازدحام مبتنی بر تأخیر، از پر شدن صف گلوگاه و وقوع Drop جلوگیری می‌کند؛ در نتیجه رفتار loss برای Flow1 و Flow2 تفاوت معناداری ندارد.



شکل ۱۷: نمودار Loss برای Flow اول از روش Vegas



شکل ۱۸: نمودار Loss برای Flow دوم از روش Vegas